

Two Strategies for HPC- QPU Integration at NCSA

Santiago Núñez-Corrales, PhD

Quantum Lead Senior Research Scientist, National Center for Supercomputing Applications
Faculty Affiliate, Illinois Quantum Information Science and Technology Center
Core Faculty, Arms Control & Domestic and International Security
University of Illinois Urbana-Champaign



**National Center for
Supercomputing Applications**

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

1. NCSA's Quantum Computing Strategy

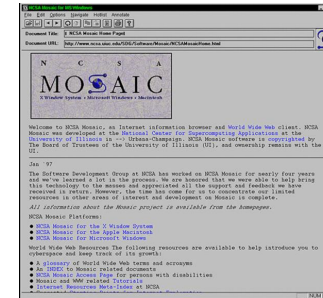


**National Center for
Supercomputing Applications**

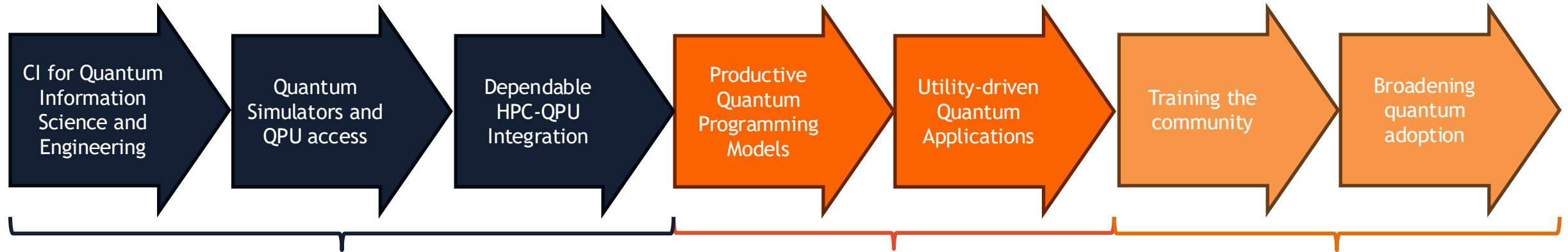
UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

National Center for Supercomputing Applications

Mission: Bring people, computing and data together to benefit society



NCSA's quantum portfolio



Expertise Areas

Simulation, networking and cyberinfrastructure

- LCCF: HPC-QPU integration
- Middleware for distributed QC
- Public Quantum Networks Software Stack
- LabDragon: quantum electronic notebook
- Quantum Simulator Resource Access (DeltaAI)

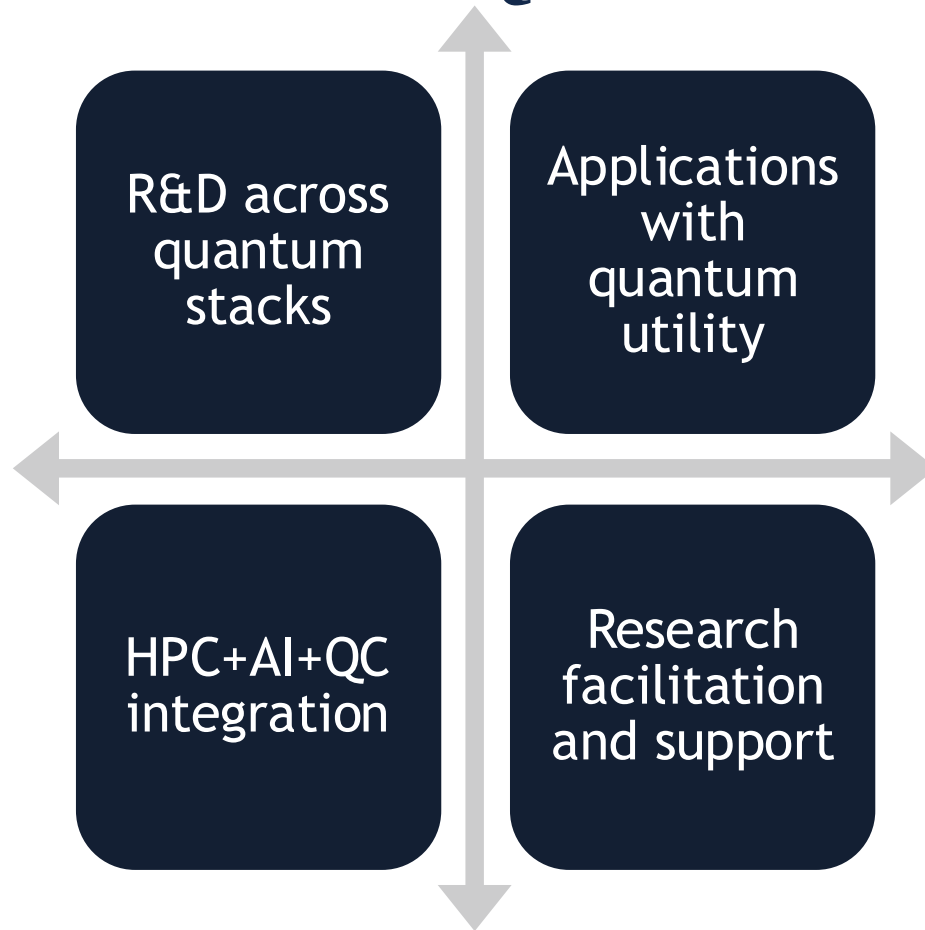
Scientific software development

- quAPL: high-level language for QC
- Next-generation quantum PLs
- Quantum causality discovery
- Classical-quantum virtual machines
- Post-quantum cryptography observatory

Training and facilitation

- 8+ workshops with QC vendors
- Partnerships with IQIIST, GCoE, CQE, IQMP
- Broad industry relations with quantum vendors and quantum-interested companies

NCSA's mission in Quantum Computing



Kernels/Libs

QML

Q-UI/UX

Applications

Quantum High Level Languages

Quantum Instruction Set Architectures

Quantum Micro/Nano programming

Quantum Circuit Execution

Quantum Circuit Compilation

Quantum Error Correction

Quantum Pulse-Level Programming

Post-Quantum
Cryptography

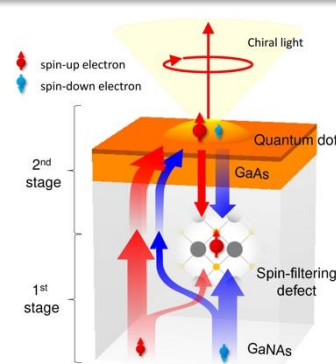
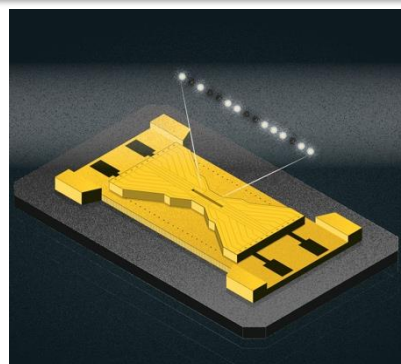
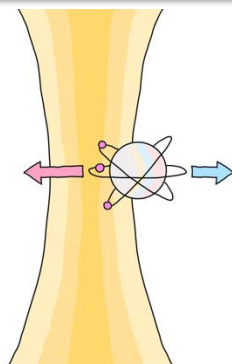
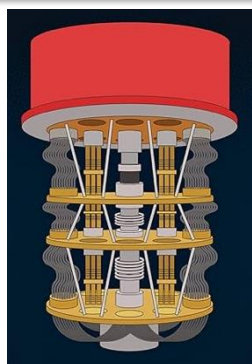
Quantum Abstract
Machines

HPC-QPU
integration

I
NCSA

Distributed
Classical-Quantum
Computing

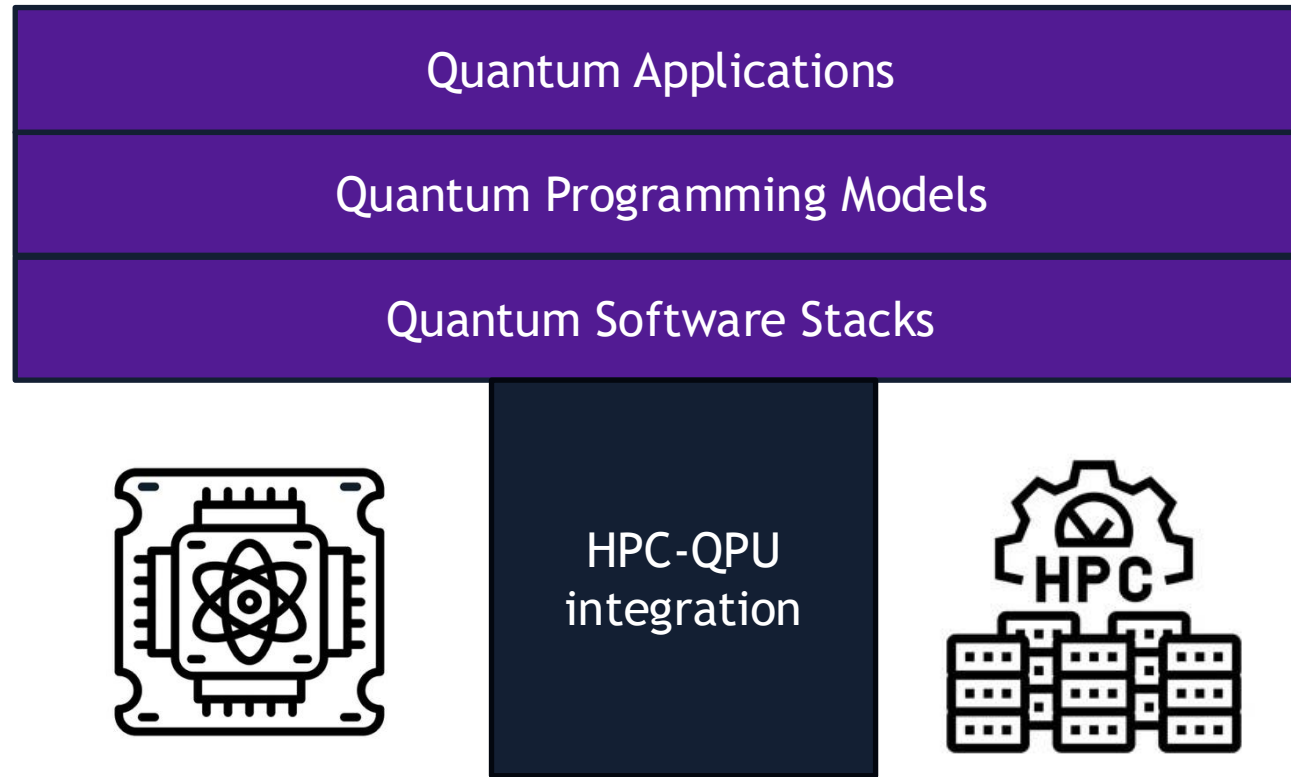
Quantum
Networks



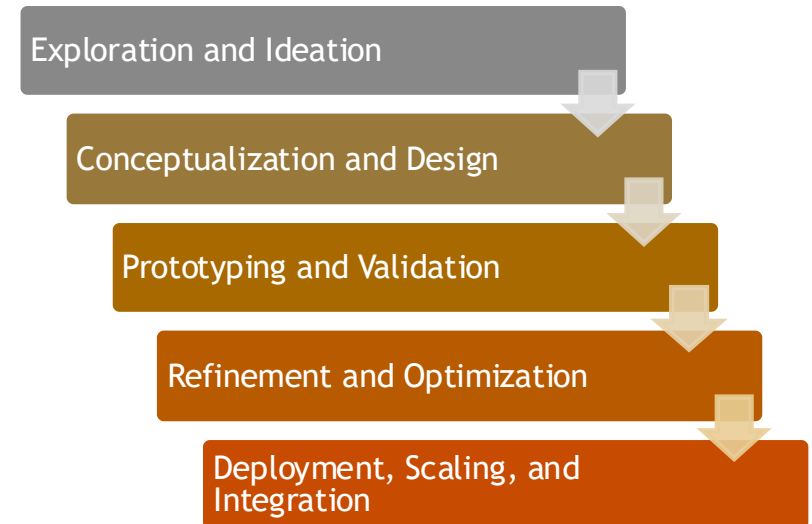
HQAN

I
IQUIST

Integrating quantum computing in non-disruptive ways



- NCSA has a history of enabling advanced compute resources to research and industrial communities
- Top-down approach to quantum: from applications to systems
- We seek to enable a maturity model for quantum applications discovery



2. LCCF: direct HPC-QPU integration

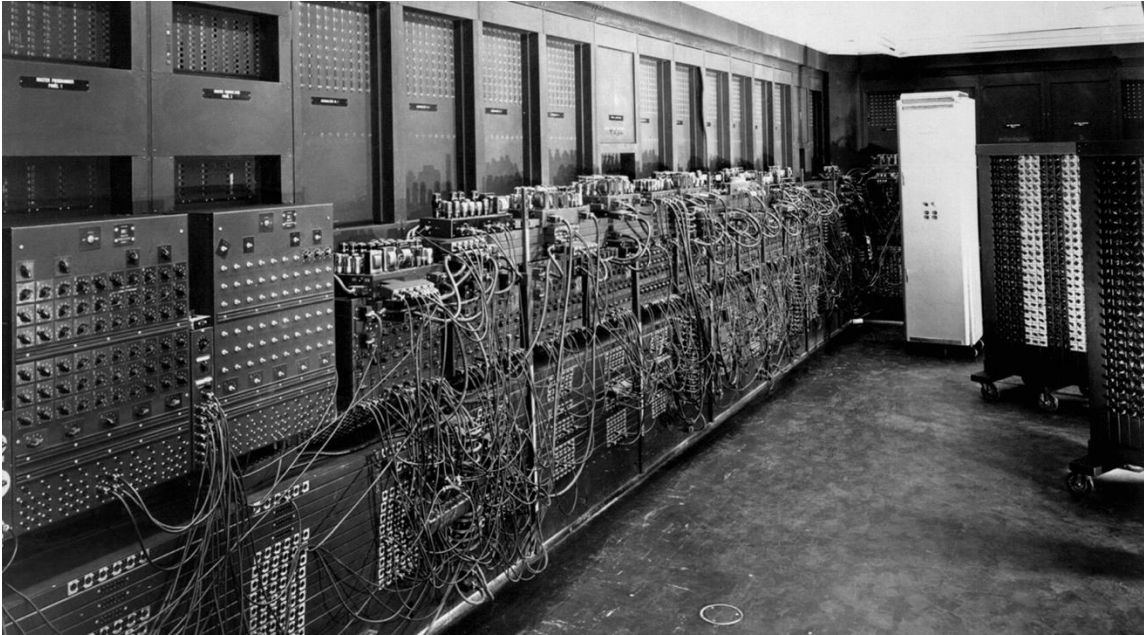


**National Center for
Supercomputing Applications**

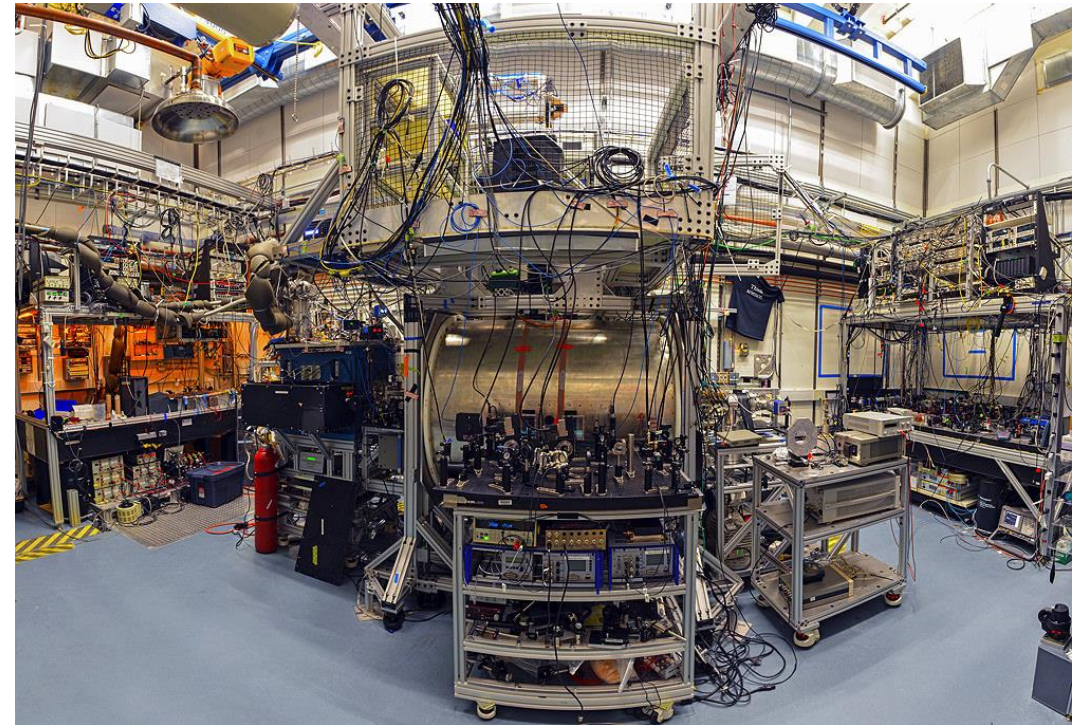
UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN



Academic hardware - Back to the 1940-1950



UPenn 1945 (ENIAC)



Harvard 2024 (HQI)

Staying at the forefront is messy. But: not all mess is unavoidable. **Experimentalists can benefit from interacting with HPC experts.**

Vendor hardware - Back to the 1950-1960



UF Gainesville 1968 (Burroughs)

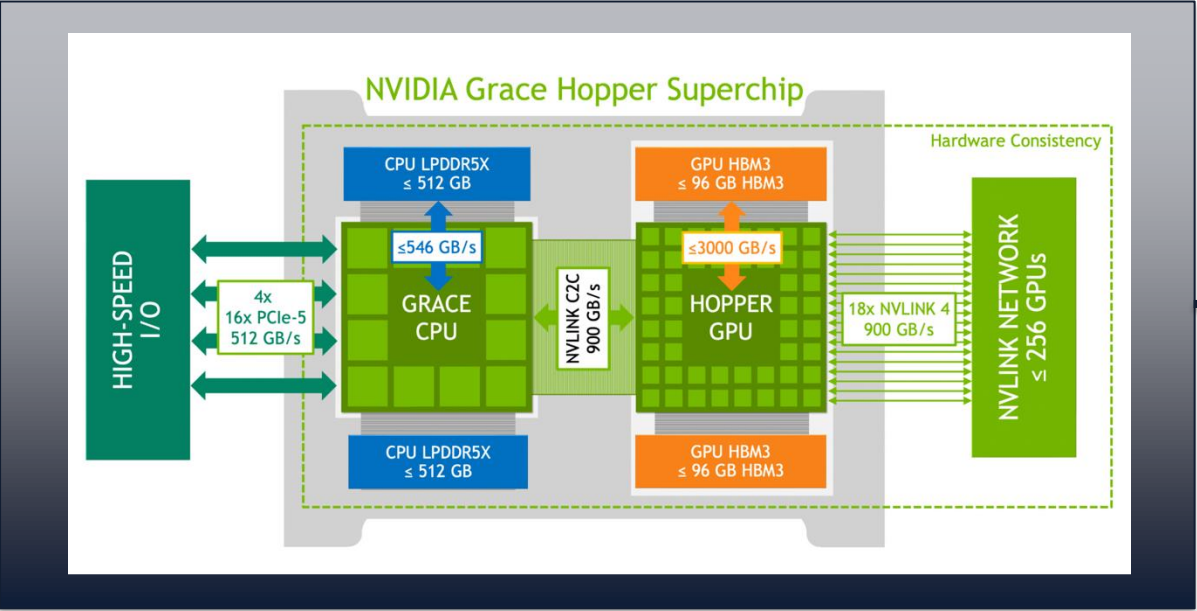


Munich Valley 2024 (IQM)

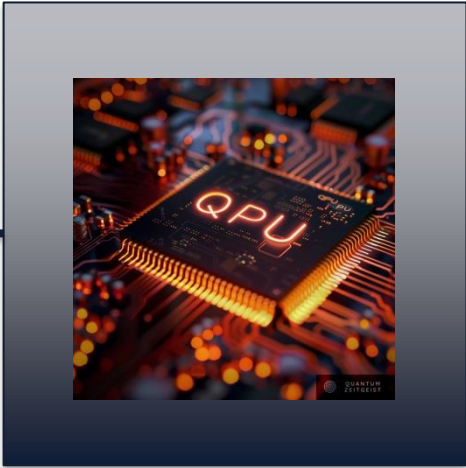
Market pressures drive innovation fast. Market pressures explain technology gaps. **HPC-QPU integration in flux.**

Leadership Class Compute Facility – LCCF (NSF)

HPC+AI @ NCSA



QPU @ IQUIST



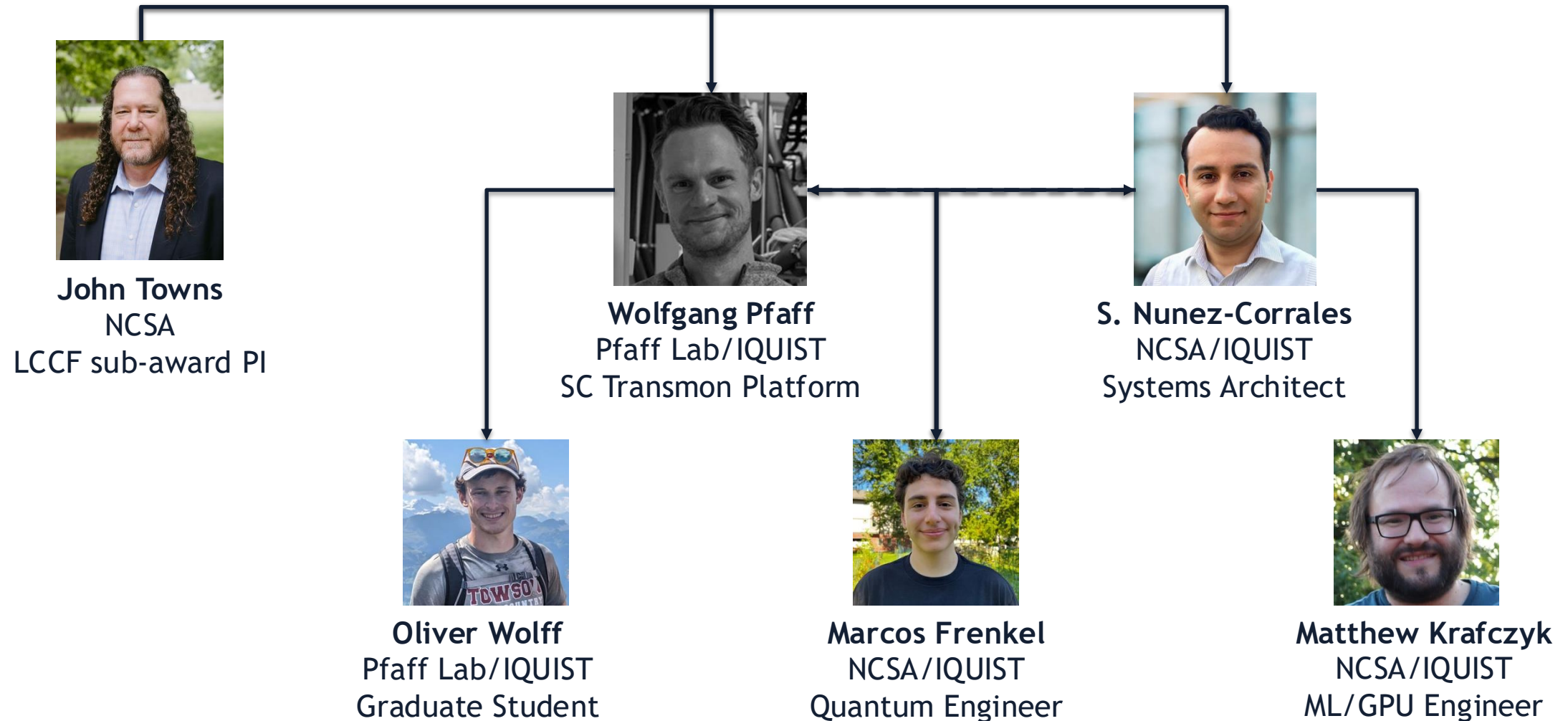
Tight HPC-AI-QPU integration	Development of quantum cyberinfrastructure	Deploy research and user access
------------------------------	--	---------------------------------

Technical Objective: operational by July 2026

Demonstrate *proof-of-principle of HPC-QPU integration* by:

- Providing a service interface
- OpenQASM-compatible + a limited number of functions
- Homegrown quantum platform (Pfaff Lab @IQUIST)
- Python library within CPU/GPU
- Applications written (possibly) by non-physicists
- Executed on NCSA-owned HPC systems hosted at NPCF

The LCCF HPC-QPU integration team



Setting up the team

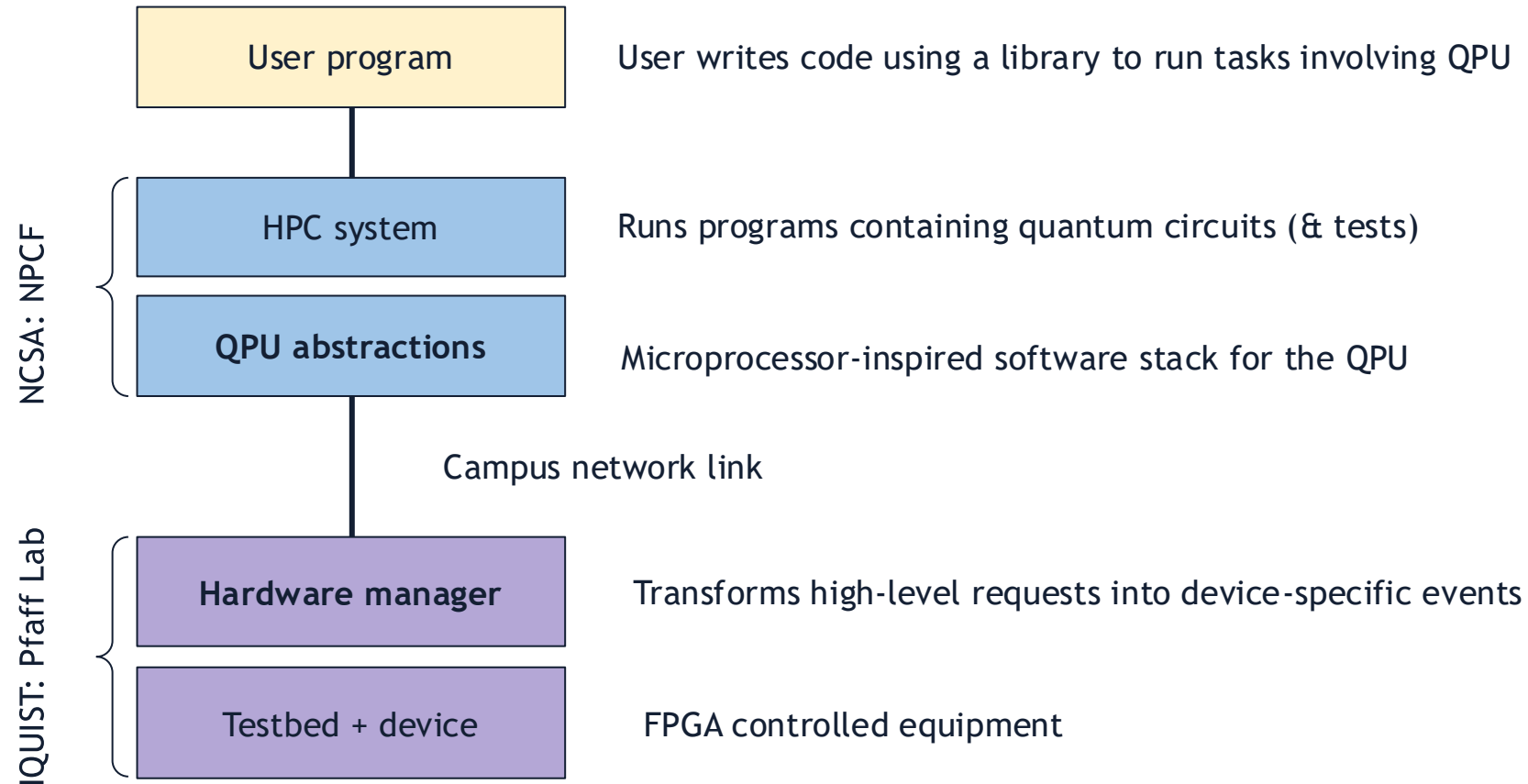
- HPC-QPU integration is interdisciplinary
- Cross-team training is essential
 - Defining terminology
 - Minimal progress metrics per field
 - Meaning of go/no-go
- 3+ months in training
 - Design as part of training
- Start of implementation after meaning is clear
- Constant revision against roadblocks: adapt and overcome

HPC system: classical computer system composed of multiple, coordinated processing elements capable of running code at scales beyond the limitations of individual user systems.

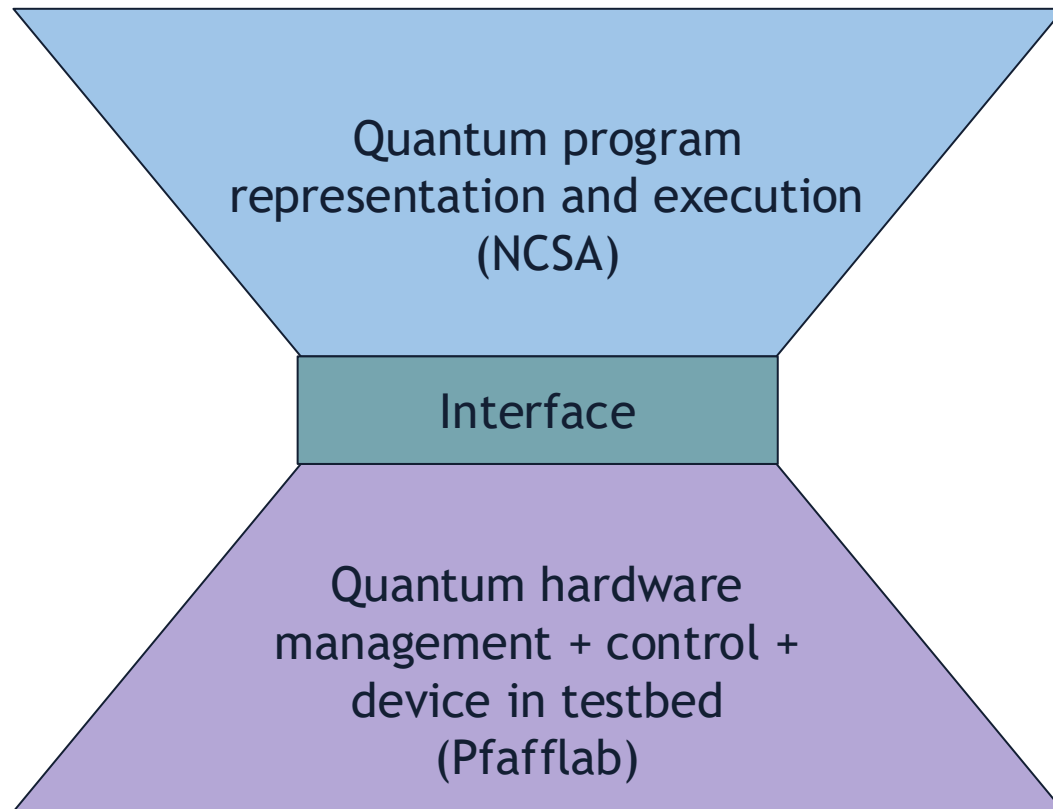
QPU system: the combination of transmon devices designed and/or manufactured at the Pfaff Lab, the instrumentation and control electronics in the IQIST testbed required to achieve quantum regimes of operation, as well as the associated control software stack.

Service interface: formal specification of standard user requests and resulting behaviors of a software system. The specification describes preconditions the state of the system must satisfy in order to exhibit correct behavior, and what deviates from it in the form of errors.

High-level view



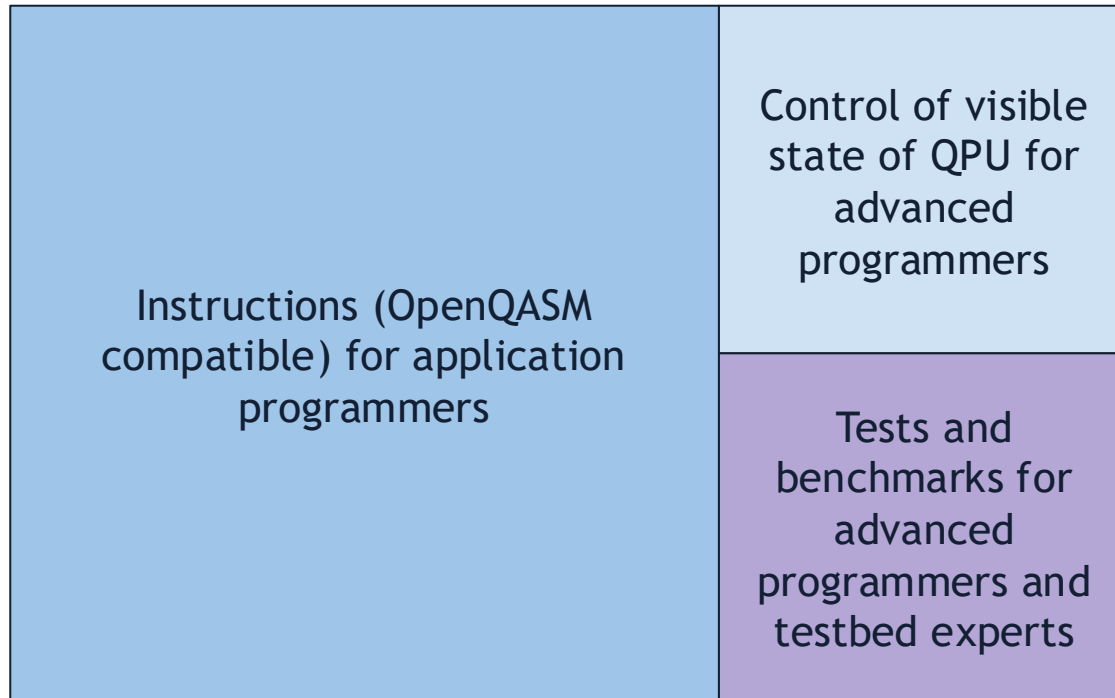
The team interface view: division of labor



Aims:

- Narrow interface, easy to compile to, easy to satisfy at control level
- Stable contract for programmers, ideally invisible and managed by services and libraries
- Security and reliability when accessing quantum hardware remote
- Full quantum device independence
- Full HPC system independence
- Reference architecture
- Transparent, inspectable implementation

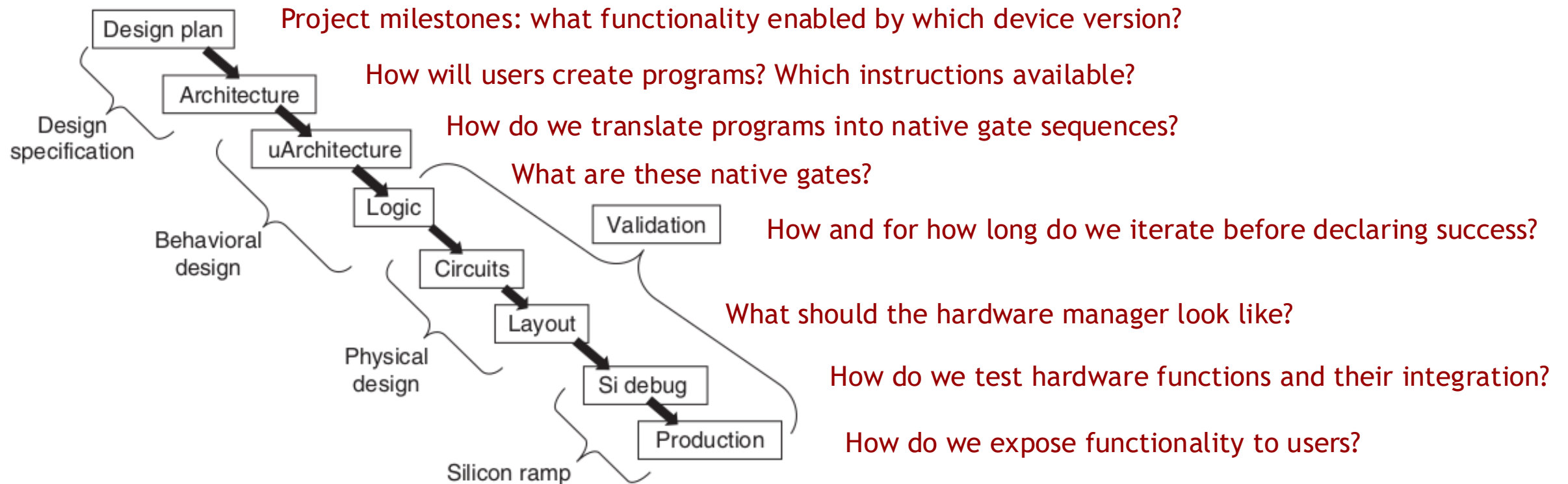
Target Instruction Set Architecture



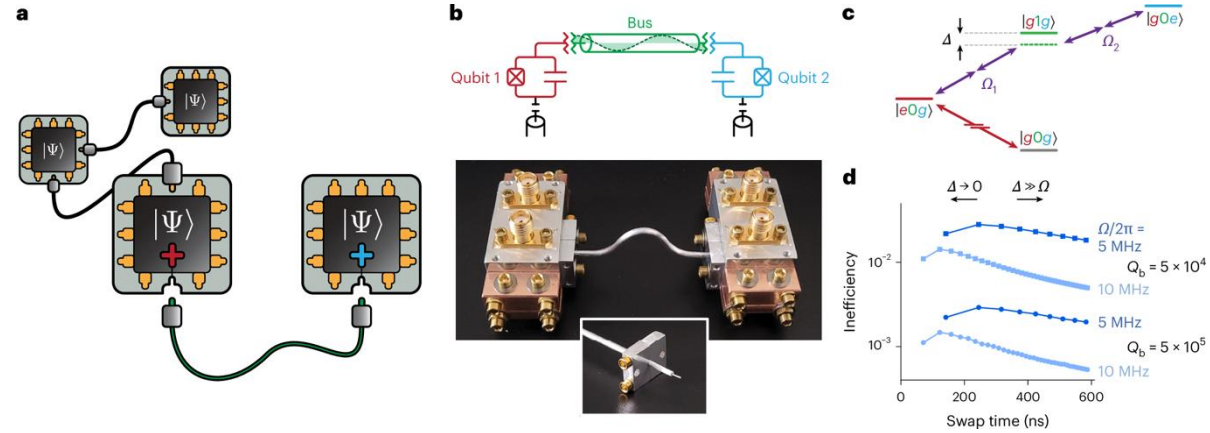
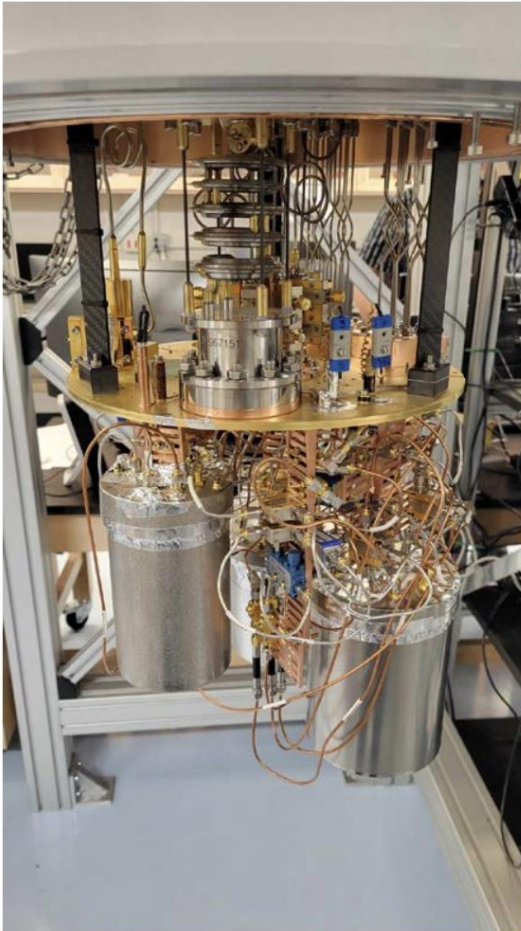
Design philosophy:

- Application programmers - OpenQASM 3.0 compatible
 - Multiple backends
 - Synthesis to other IRs
- System programmers and hardware experts
 - Full control of QPU state abstraction
 - Automated setup initialization
- Expert users, QISE scientists
 - Fine-grained interrogation of quantum hardware (e.g., power Rabi, qubit spectroscopy)
 - Automated multiqubit calibration

More borrowing: microprocessor design



Quantum hardware: Pfaff Lab



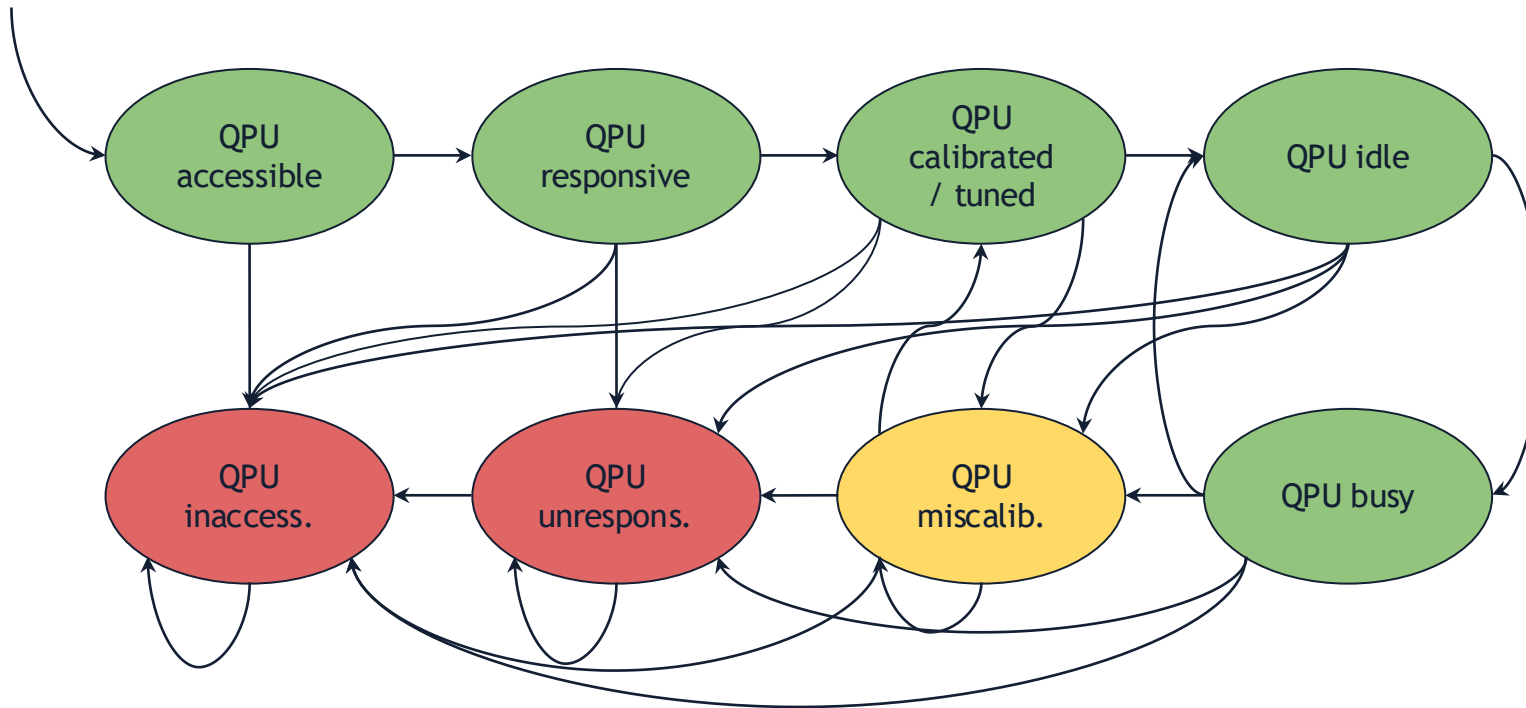
Mollenhauer, M., Irfan, A., Cao, X., Mandal, S. and Pfaff, W., 2025. A high-efficiency elementary network of interchangeable superconducting qubit devices. *Nature Electronics*, pp.1-10.

LCCF:

- Superconducting transmon qubits (Al/Si)
- Spin chain device
- Native gates: X, Y, sqrt[iSWAP]

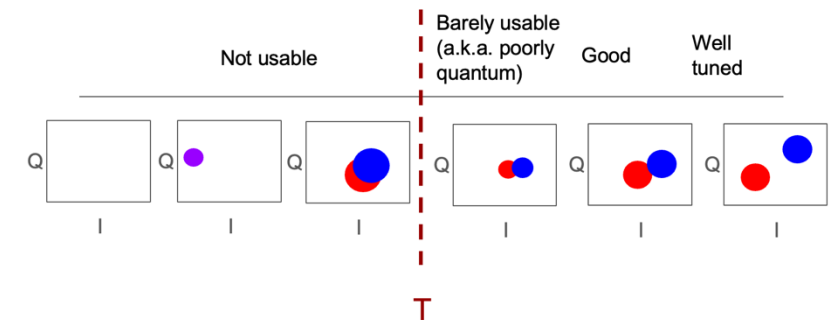


The QPU state abstraction

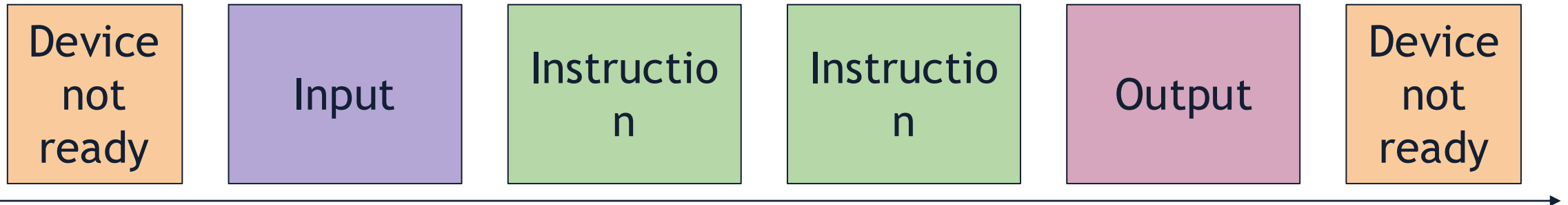


Core aim:

- Abstract state over whole quantum device
- Informative for application and systems programmers
- Informative to trace quantum hardware error failures
- Automatable and inspectable



Overall lifetime of a program: the ISA view

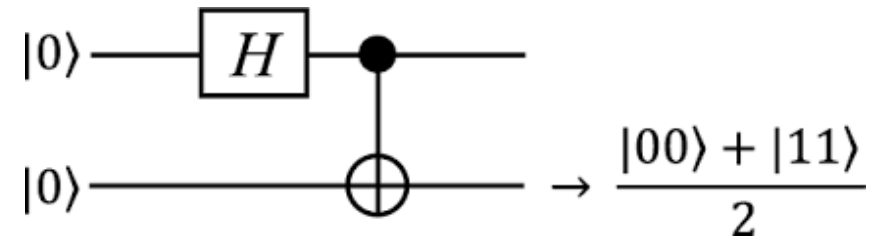


```
OPENQASM 2.0;  
include "qelib1.inc";
```

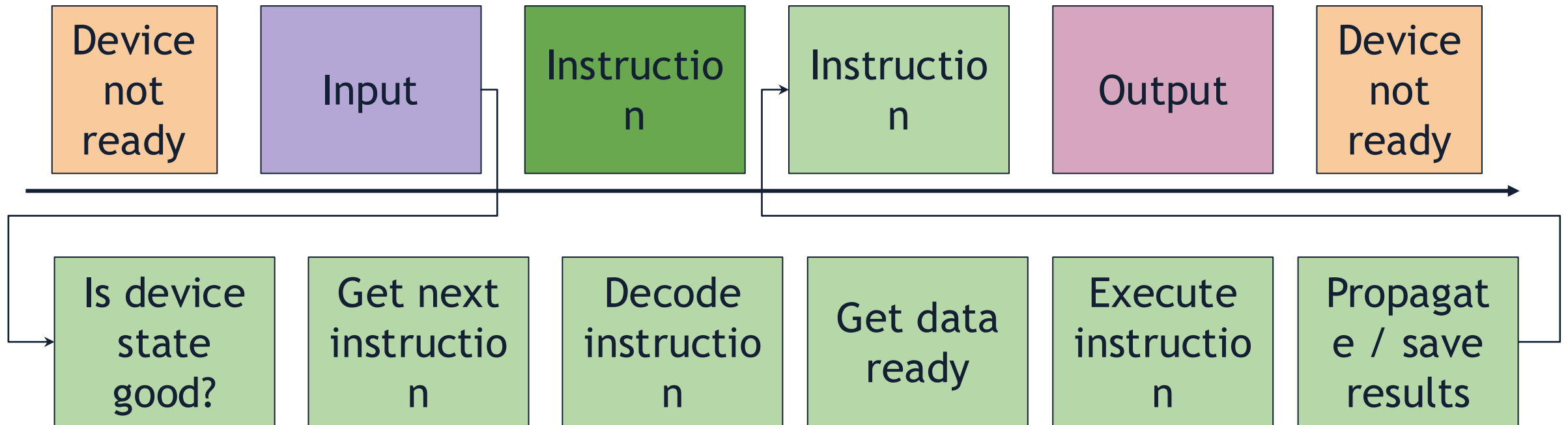
```
qreg q[2];  
creg c[2];
```

```
h q[0];  
cx q[0], q[1];
```

```
measure q -> c;
```

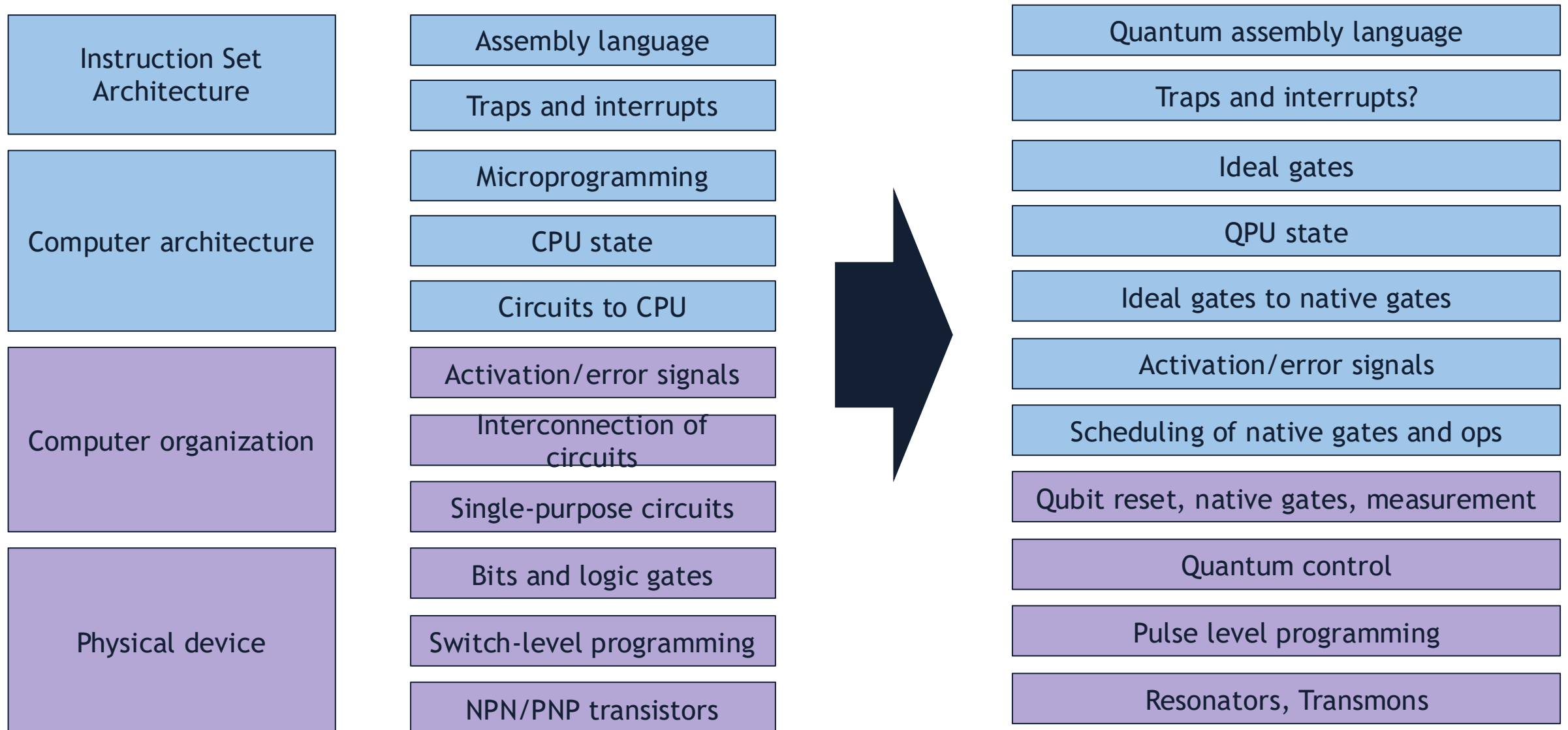


Overall lifetime of a program: microprogramming

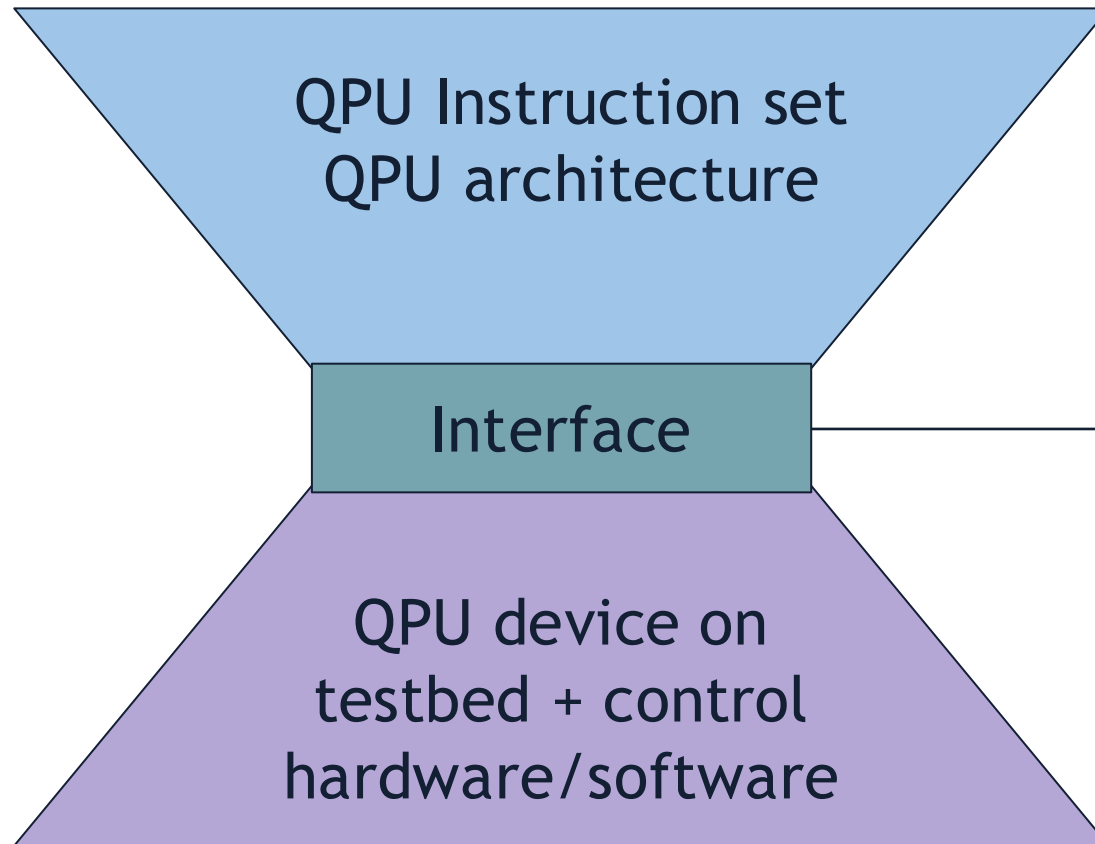


LCCF: the hardware manager will perform the execute instruction using hardware functions, we will perform the decode step from above the narrow waist

Expanding the boxes - quantum equivalents? Kind of.



Proposed “narrow waist”

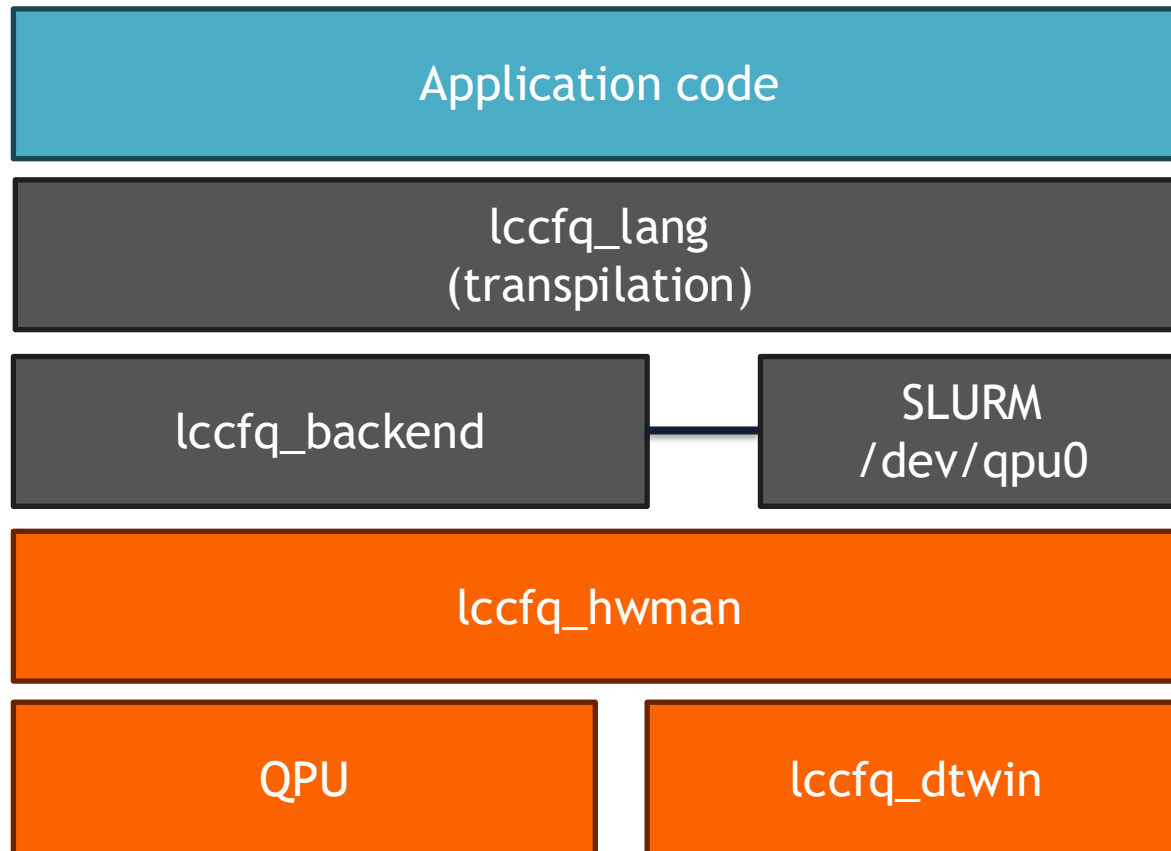


Quantum assembly language
Traps and interrupts?
Ideal gates
QPU state
Ideal gates to native gates
Activation/error signals

Scheduling/mapping of native gates

Qubit reset, native gates, measurement
Quantum control
Pulse level programming
Resonators, Transmons

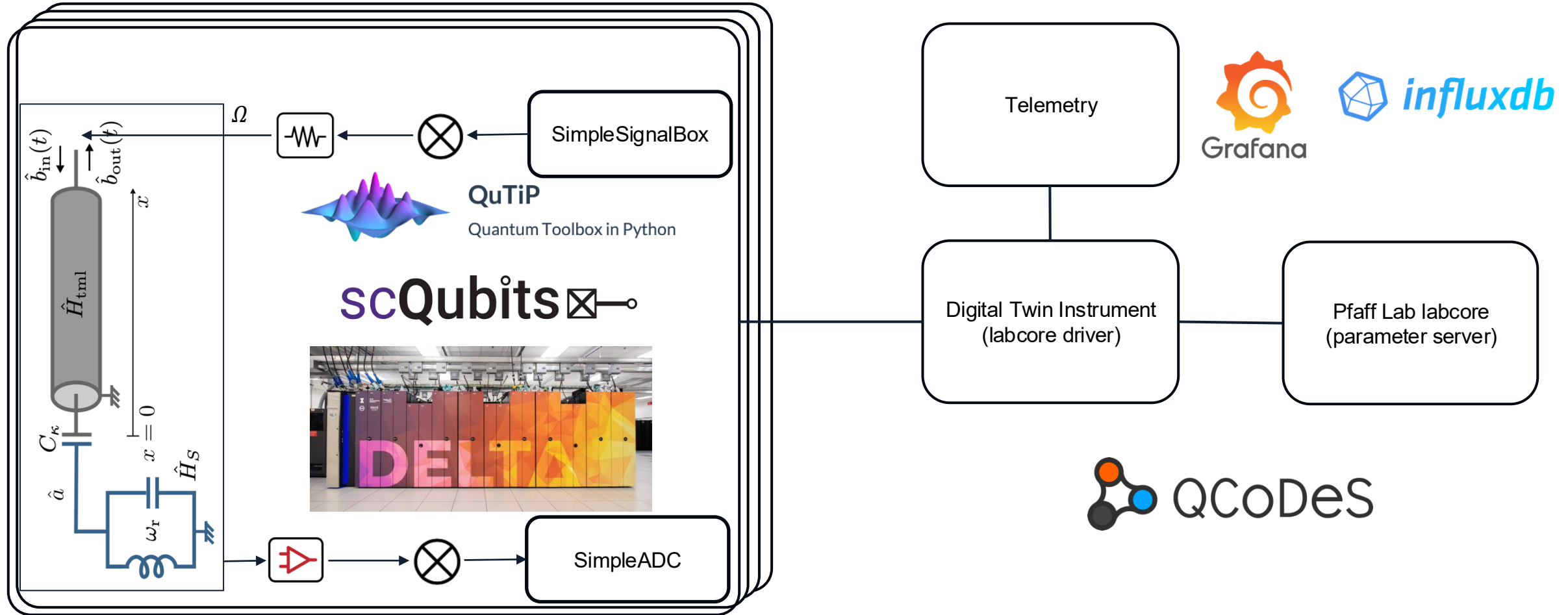
The actual software pieces



Lessons borrowed from history of computing:

1. Provide end-users with stable mechanism to specify classical programs with heterogeneous processing units involved (CPU, GPU, QPU)
2. Separate instruction denotation from execution mechanics
3. Orchestrate individual hardware functions to vary the behavior of end-user instructions
4. Separate hardware functionality from specific devices available
5. Isolate hardware improvements in performance/quality from stable functionality

Digital twin: progress when device is down



```

In [ ]: import json
import matplotlib.pyplot as plt
from dtwin.physics.devices import TLineTransmonDevice, DeviceType
from dtwin.physics.dynamics import Dynamics
from dtwin.instruments.pulses import Pulse
from dtwin.instruments.signals import Multiplier
path = '../dtwin/conf/tl-transmon.json'
with open(path, 'r') as f:
    params = json.load(f)

In [ ]: # Define the device according to the params json
device = TLineTransmonDevice(params['name'], params)

# Define the pulses that will be used to drive the device
pulses = []
pulse_types, pulse_params = params['signals']['pulses'].keys(), params['signals']['pulses'].values()

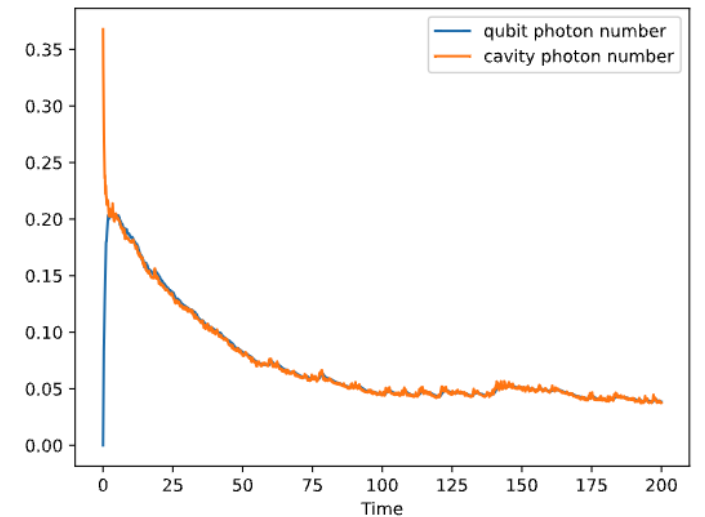
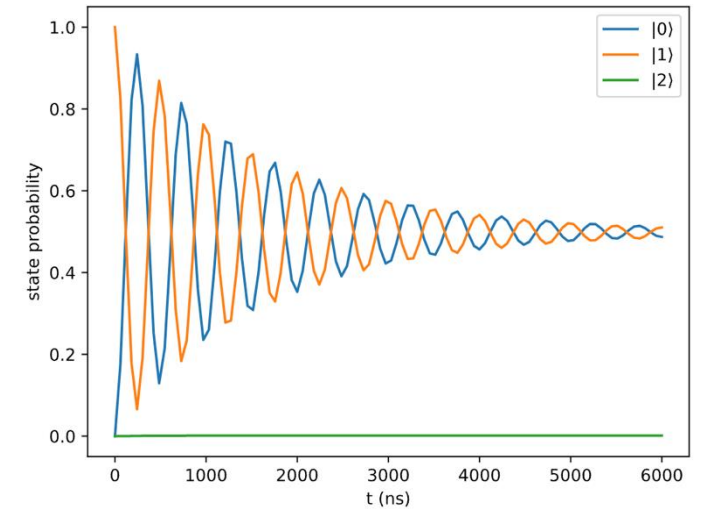
for pulse_type, pulse_param in zip(pulse_types, pulse_params):
    p = Pulse.create(pulse_type, pulse_param)
    pulses.append(p)

# Modulate the pulses
pulse = Multiplier.modulate_signals(pulses)

# Create an instance of the dynamics class
dynamics = Dynamics()
# Set the device of interest
dynamics.set_device(device)
# Set other relevant parameters for the system
dynamics.set_from_params(params)
# Add the drive term using the pulse defined before
dynamics.add_drive(pulse)
# Perform a shot
res = dynamics.do_shot()

In [ ]: plt.plot(res.times, res.expect[0], label='qubit photon number')
plt.plot(res.times, res.expect[1], label='cavity photon number')
plt.legend()
plt.xlabel('Time')

```



3. Quantum Programming Abstractions: indirect HPC-QPU integration



**National Center for
Supercomputing Applications**

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

QCE 22

Circuits are not very
good for writing
quantum algorithms

Oh no, they
aren't. Let's talk
with more people
about it



<https://www.mustythoughts.com/beyond-circuits>



QCE 23

Quantum Computing Market Success Requires an Application-level Programming Model that Delivers Performance

IEEE Quantum Week 2023 conference

Hyatt Regency Bellevue on Seattle's Eastside

900 Bellevue Way NE, Bellevue, Washington, United States, 98004-4272

Room: Regency B

Date: Tue, Sep 19, 2023

Time: 10:00-16:30 Pacific Time (PDT) — UTC-7

Duration: 4.5 hours (3 x 1.5 hours)



QCE 24

An Abstraction Hierarchy Toward Productive Quantum Programming

Olivia Di Matteo
Electrical and Computer Engineering
The University of British Columbia
Vancouver, Canada
olivia@ece.ubc.ca
ORCID 0000-0002-1372-7706

Santiago Núñez-Corrales
NCSA/IQUIST
University of Illinois Urbana-Champaign
Urbana, IL, USA
nunezco2@illinois.edu
ORCID 0000-0003-4342-6223

Michał Stęchły
Musty Tech
Oshawa, ON, CA
michal@mustythoughts.com
ORCID 0000-0002-2391-9664



Steven P. Reinhardt
Transform Computing, Inc.
Eagan, MN, USA
ORCID 0000-0003-4355-6693

Tim Mattson
Human Learning Group
Ocean Park, WA, USA
ORCID 0000-0002-6106-8717



QCE 25

Programming Abstractions Workshop @...

Friday, 5 September 2025

Enabling Programming Abstractions for More Effective Quantum Development

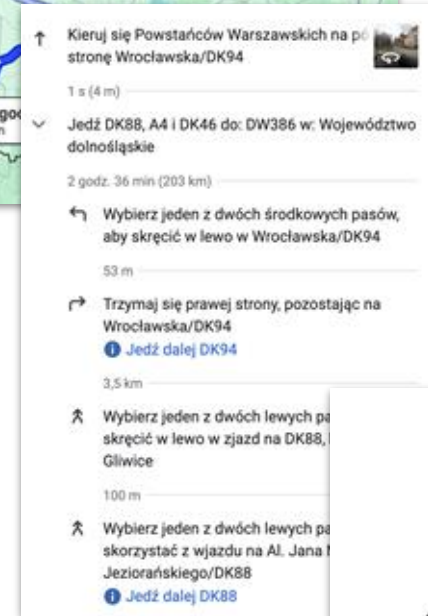
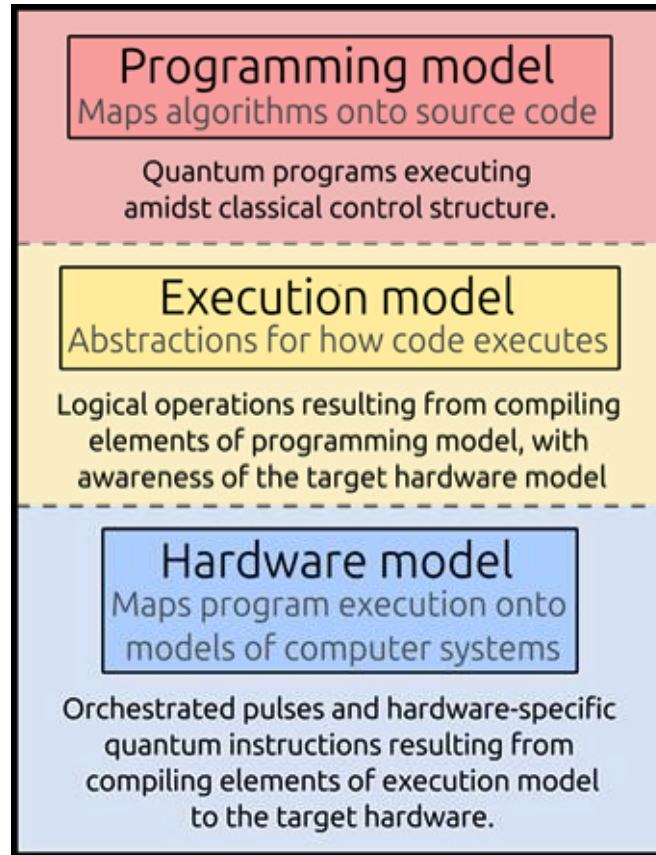
at IEEE Quantum Week
ABQ Convention Center
Albuquerque, NM, US

An effective, accessible abstraction hierarchy has made using and programming classical computers possible for people across all disciplines. It is our hope that the same can be achieved for quantum computing. However, current quantum programming languages and frameworks often require users to have PhD-level knowledge of information theory, mathematics, physics, and/or chemistry.

At the applications level, the prevalence of gate-based reasoning to develop quantum algorithms hampers non-experts in delving into the domain.

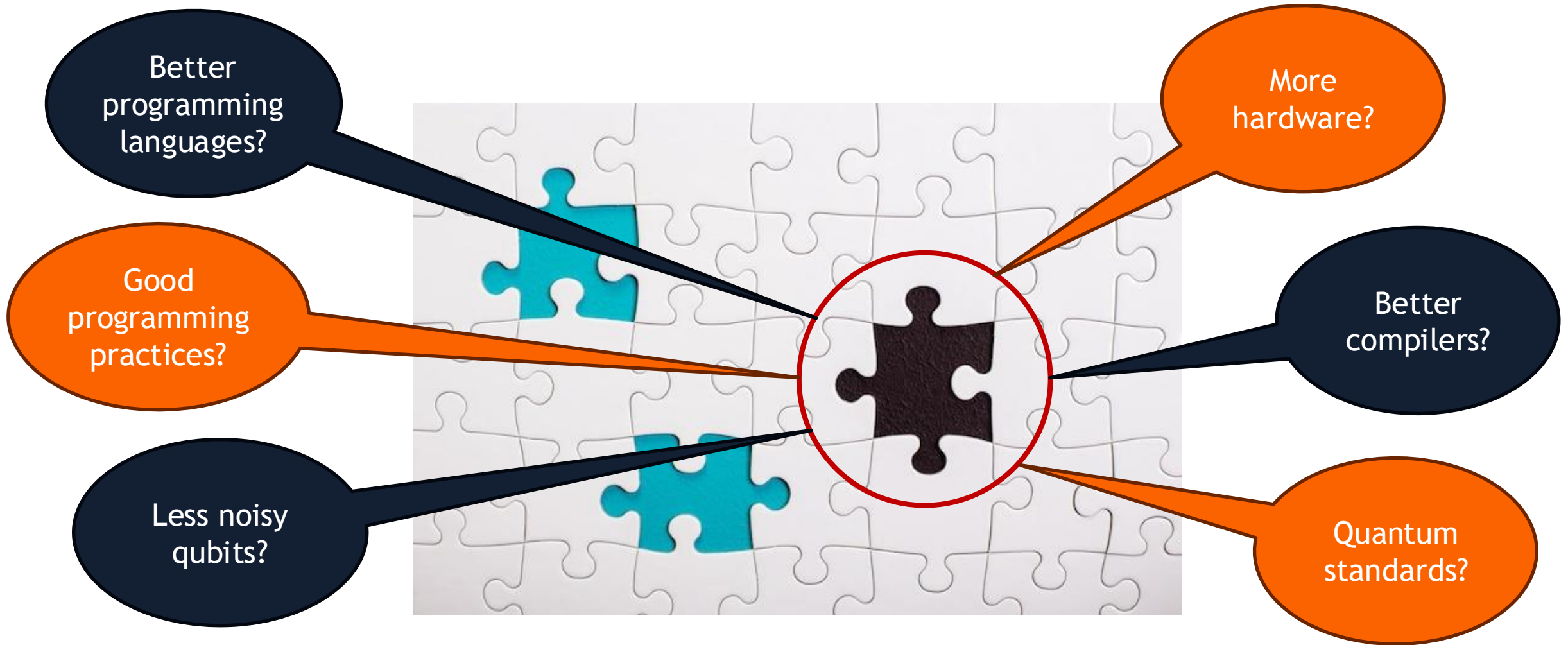


Abstractions mediate effectiveness



Di Matteo, O., Núñez-Corrales, S., Stęchły, M., Reinhardt, S.P. and Mattson, T., 2024, September. An abstraction hierarchy toward productive quantum programming. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)* (Vol. 1, pp. 979-989). IEEE.

But: something really important is missing...



What is an “*abstract machine*”?

structure and
transformations

computable
functions

a *model of computation* that specifies what data can be input, output, and stored, and provides a *semantics of operations* that can be performed on that data

from form to
execution effects



What is an “*instruction*”?

interlocking
building blocks

minimally useful
representation

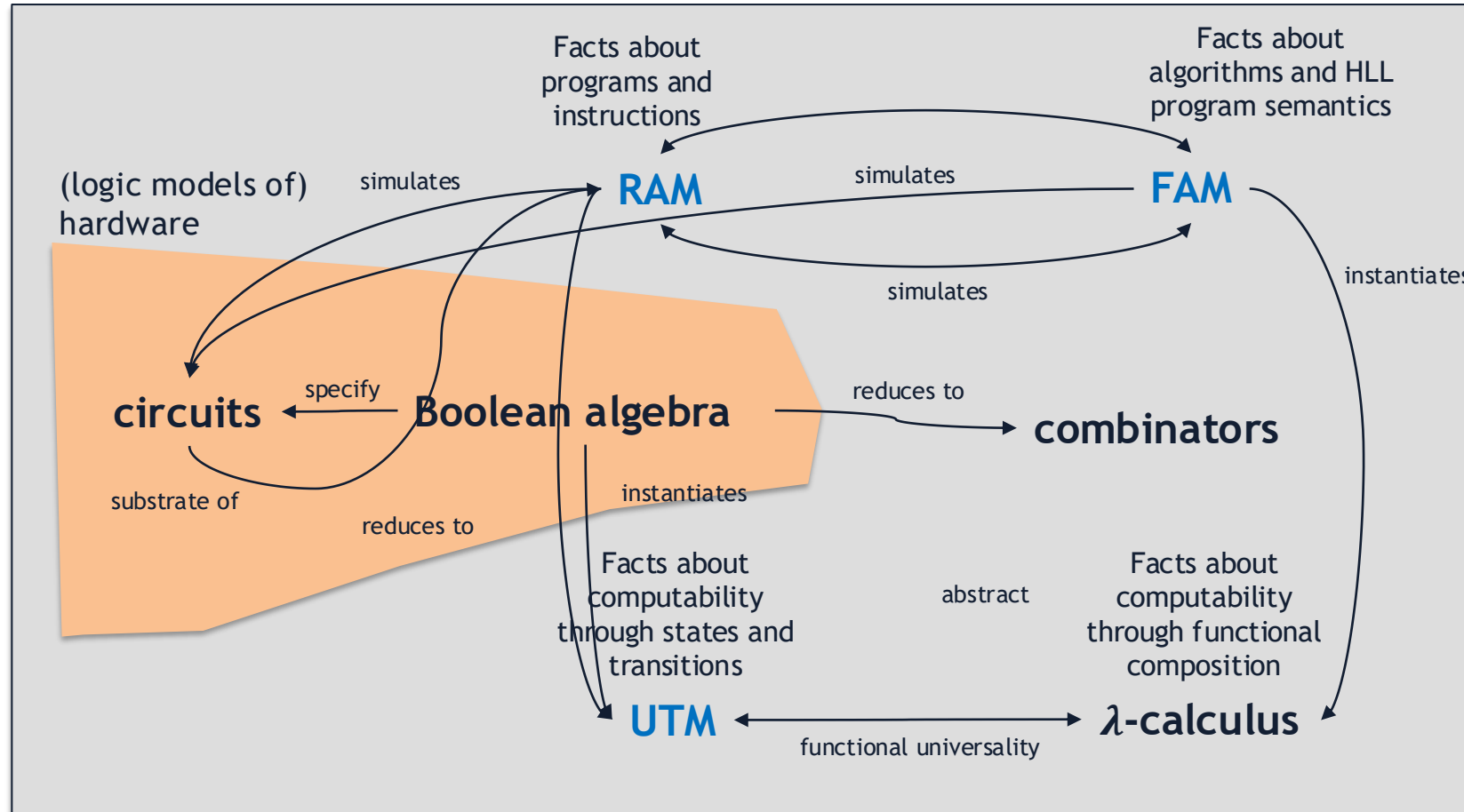
an *atomic*, *composable* procedural
abstraction

no need to
decompose further

part of an
algorithmic recipe



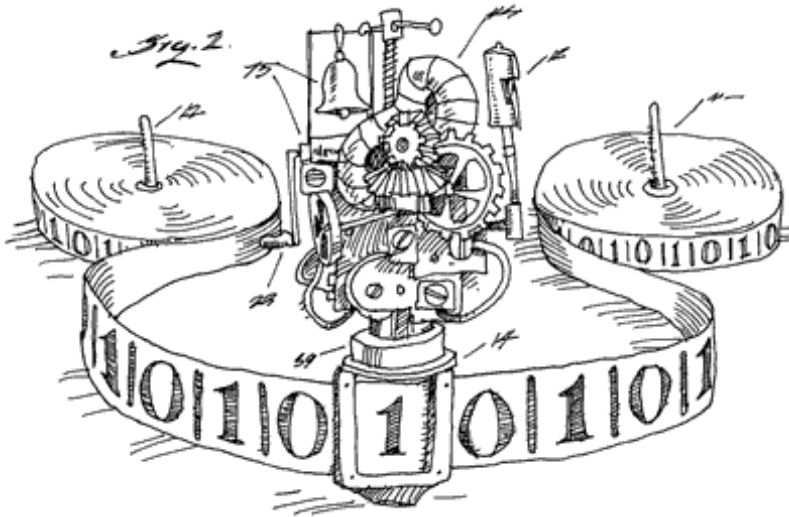
Abstract machines serve different purposes



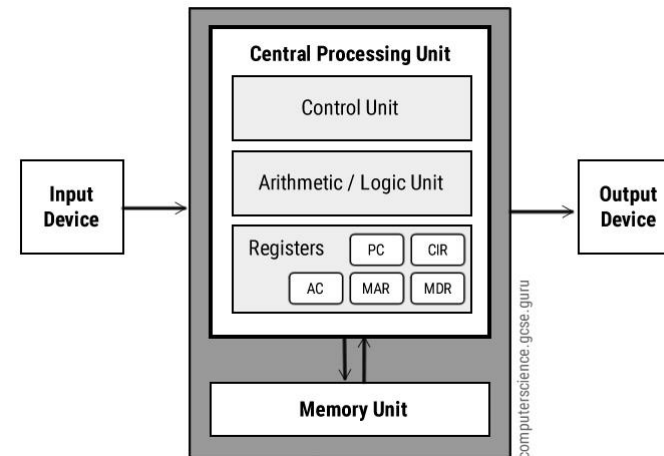
But: not all lead to productive programming or scalable hardware



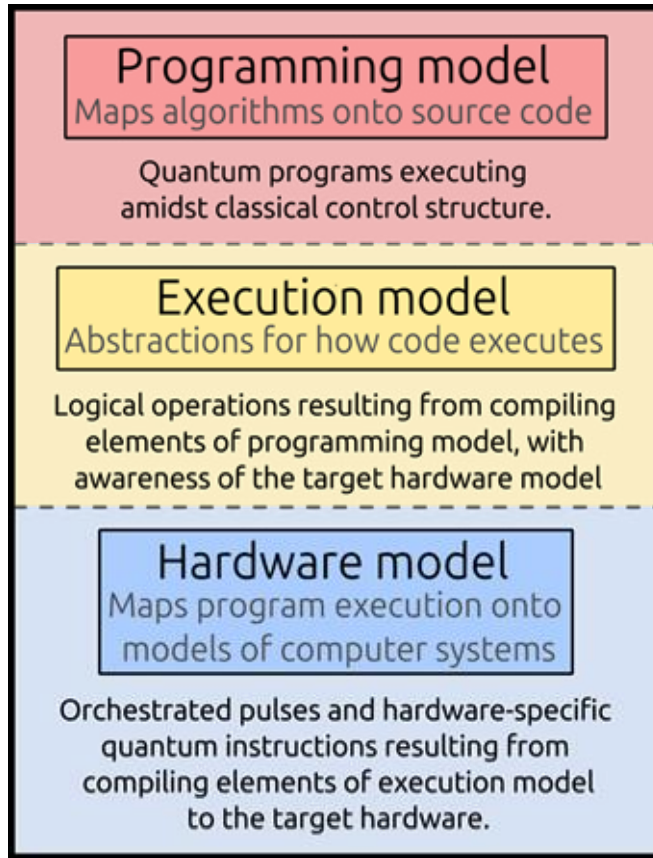
Turing machines: which functions are computable?



Von Neumann machines: how do we build programmable electrical computers?



Abstract machines = execution model



What makes then a good abstract machine?

- Stable contract between programs and hardware
- Instruction set architecture fits in the mind of the programmer
- Instantiated via hardware implementation
- Independent of all hardware implementations

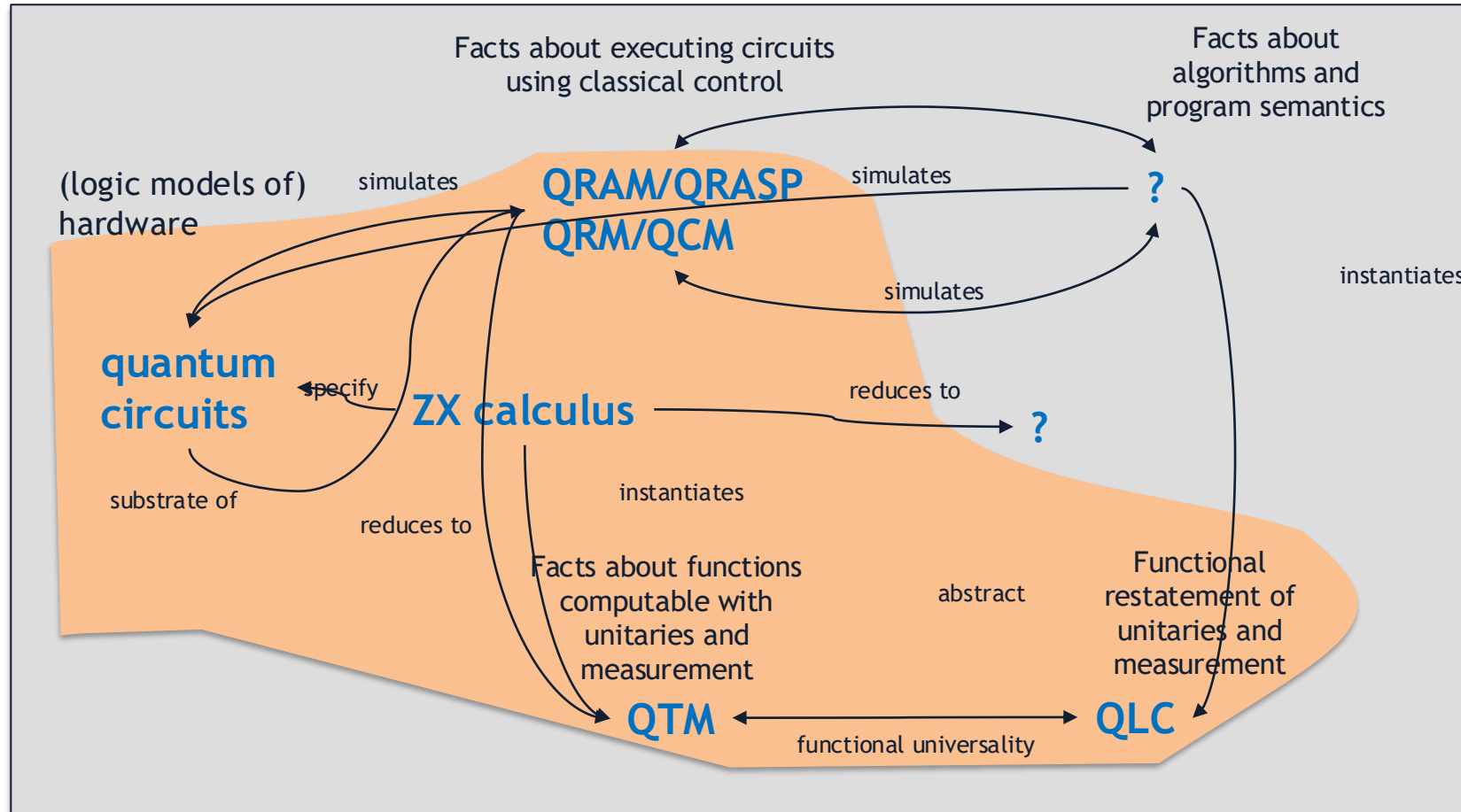
What about today's QAMs?

Hint: they are not great

- Multiple ways to define quantum abstract machines
- All of them involve circuits
- Some are realistic, some involve hardware
- Varying degrees of presence and sophistication of control flow
- Three large families
 - Quantum Turing Machines
 - Von Neumann-like Machines
 - Quantum Lambda Calculus



All quantum abstract machines so far are only great circuit execution machines



Evaluation time!

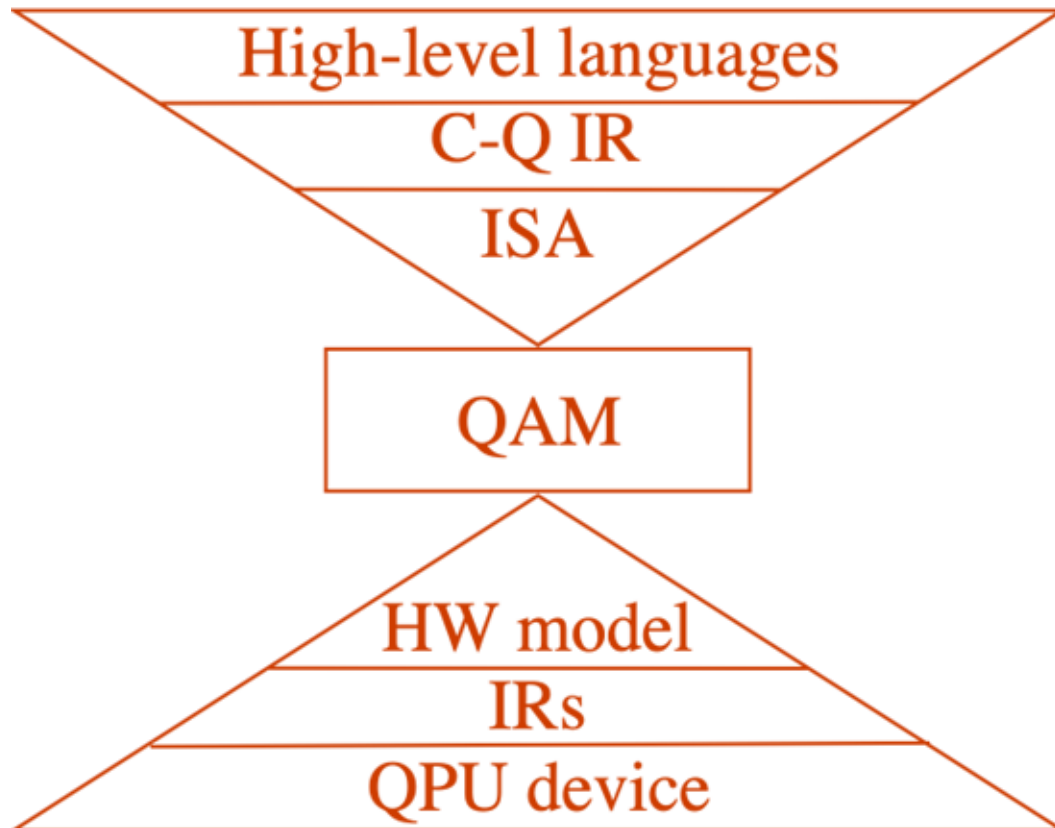
TABLE I
ANALYSIS OF PREVAILING QUANTUM ABSTRACT MACHINES

Criterion	Description	QTM [19]	QRAM [19]	QRASP [19]	QRM [20]	QCM [21]	QLC [37]
1	Turing-complete & universal	✓	✓	✓	✓	✓	✓
2	Finite symbolic state				✓	✓	
3	Symbolic denotational semantics						
4	Representation-independent data types						
5	Stable instruction set architecture						
6	Verifiable formal content	✓	✓	✓	✓	✓	✓
7	Classical-quantum regularity	✓			✓ [†]	✓ [†]	
8	Compact instruction representation		✓	✓	✓	✓	✓
9	Degeneracy of implementation	✓	✓	✓	✓	✓	✓
10	Predictable procedural composability		✓	✓	✓	✓	
11	Intrinsic ensemble semantics	✓	✓	✓			
12	Resource-constructible functions	✓	✓	✓	✓	✓	✓
13	Standard instruction cycle		✓	✓	✓	✓	
14	Classical control flow			✓	✓	✓	✓
15	Quantum/hybrid control flow				✓	✓	
Total		6✓	8✓	9✓	11✓	11✓	6✓

[†] partial satisfaction due to explicit mention of unitary gates



If we found the right machine...

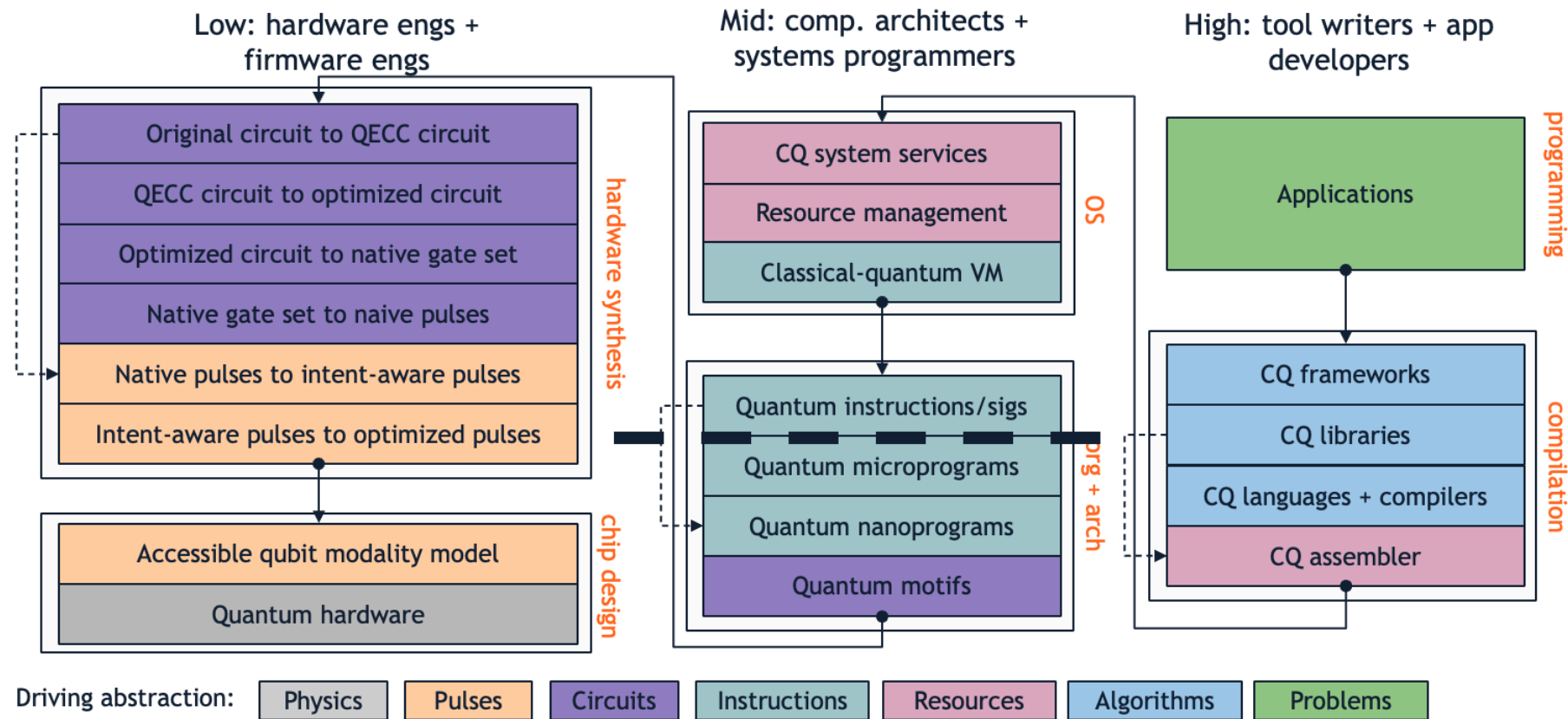


- Quantum programming accessible without PhDs in physics
- Quantum computational complexity would not equate to circuit complexity
- New control flow abstractions
- Quantum systems programming well established
- Quantum hardware synthesis != quantum compilation
- True program portability

...



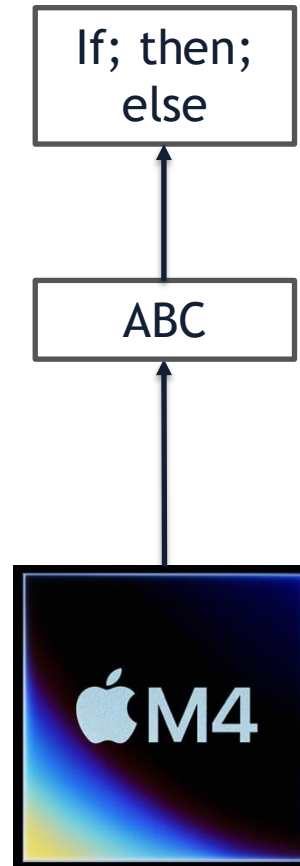
Vision – Full HPC-QPU integration



This architecture establishes new foundations for future utility-scale FTQC architectures by recognizing that *qubit technologies are not the main factor explaining future scalability of data-center-scale systems in the long term, abstractions are.*

Vision – The Challenge

- Need to break the intuition that programming models should scale with hardware resources
- We rather need **scale-free programming models** w.r.t. qubit/gate counts
- Separation of **manufacturability** from **programmability**
- We've been here before (i.e. VLSI), we will need to get there again (i.e., FTQC)
- This requires separating the denotation of quantum programs from compilation into executable physical circuits, from their execution during runtime
- **Quantum programs that fit in the mind of application writers accelerate utility**
- **Number of successful applications inversely proportional to number of QISE PhDs needed to develop them**



$O(1)$ control flow patterns

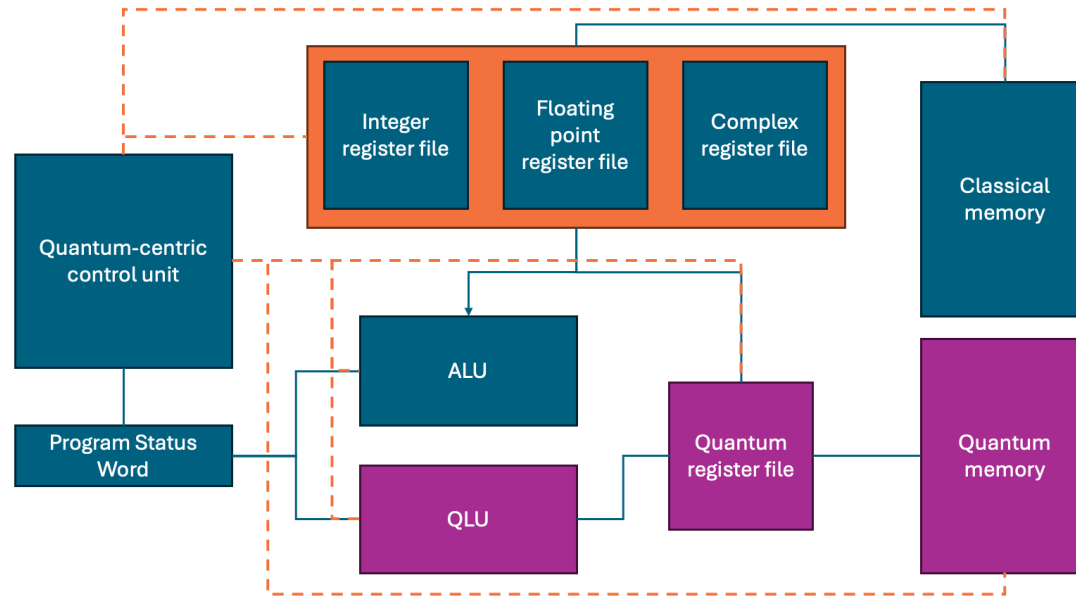
10^3 instructions
 $\sim 10^2$ used 90% of time

$\sim 2.8 \times 10^{10}$ transistors

7-8 orders of magnitude

Abstract machines organize complexity into event schedules with varied hardware

A classical-Quantum Virtual Machine



- High level assembly programs made of common building blocks across classical quantum algorithms
- Ensemble-based quantum instruction segments where number of shots is inferred, not given, registers can be ensembles
- Instructions carry annotations with them across resulting abstract circuits
- Hybrid instructions mediate between classical and quantum information
- Classical instructions as in existing microprocessors

reserved	trap	hybrid	quantum	classical																																																
<table><tr><th>Flag</th><th>Meaning</th></tr><tr><td>HT</td><td>Halt</td></tr><tr><td>TI</td><td>Interrupt</td></tr></table>	Flag	Meaning	HT	Halt	TI	Interrupt		<table><tr><th>Flag</th><th>Meaning</th></tr><tr><td>H</td><td>Hybrid block</td></tr><tr><td>KF</td><td>Forked path</td></tr><tr><td>RF</td><td>Merged path</td></tr><tr><td>TF</td><td>C/Q type cast</td></tr><tr><td>VF</td><td>Converged</td></tr></table>	Flag	Meaning	H	Hybrid block	KF	Forked path	RF	Merged path	TF	C/Q type cast	VF	Converged	<table><tr><th>Flag</th><th>Meaning</th></tr><tr><td>QF</td><td>Conditional</td></tr><tr><td>LF</td><td>Low fidelity</td></tr><tr><td>MF</td><td>Measure</td></tr><tr><td>FF</td><td>Fault</td></tr><tr><td>EF</td><td>Entangled</td></tr><tr><td>IF</td><td>Interfered</td></tr><tr><td>SF</td><td>Superposed</td></tr><tr><td>UF</td><td>Undefined</td></tr></table>	Flag	Meaning	QF	Conditional	LF	Low fidelity	MF	Measure	FF	Fault	EF	Entangled	IF	Interfered	SF	Superposed	UF	Undefined	<table><tr><th>Flag</th><th>Meaning</th></tr><tr><td>ZF</td><td>Zero</td></tr><tr><td>NF</td><td>Negative</td></tr><tr><td>CF</td><td>Carry</td></tr><tr><td>OF</td><td>Overflow</td></tr><tr><td>XF</td><td>Undefined</td></tr></table>	Flag	Meaning	ZF	Zero	NF	Negative	CF	Carry	OF	Overflow	XF	Undefined
Flag	Meaning																																																			
HT	Halt																																																			
TI	Interrupt																																																			
Flag	Meaning																																																			
H	Hybrid block																																																			
KF	Forked path																																																			
RF	Merged path																																																			
TF	C/Q type cast																																																			
VF	Converged																																																			
Flag	Meaning																																																			
QF	Conditional																																																			
LF	Low fidelity																																																			
MF	Measure																																																			
FF	Fault																																																			
EF	Entangled																																																			
IF	Interfered																																																			
SF	Superposed																																																			
UF	Undefined																																																			
Flag	Meaning																																																			
ZF	Zero																																																			
NF	Negative																																																			
CF	Carry																																																			
OF	Overflow																																																			
XF	Undefined																																																			

Group	Instructions
Classical: Integer	CL:LOAD, CL:STORE, CL:ADD, CL:SUB, CL:MUL, CL:DIV, CL:MOD, CL:POW, CL:CMP, CL:INC, CL:DEC
Classical: Float	CL:FADD, CL:FSUB, CL:FMUL, CL:FDIV, CL:FPOW, CL:FCMP
Classical: Complex	CL:CADD, CL:CSUB, CL:CMUL, CL:CDIV
Classical: Control	CL:JMP, CL:IF, CL:CALL, CL:RET, LABEL, HALT
Quantum	QPREP, QKERNEL, QFOURIER, QORACLE, QPHASEKICK, QDIFFUSE, QENTANGLE, QSWAPNET, QWALKSTEP, QREFLECT, QPHASEAMP, QGROVER, QOBSERVE
Hybrid	HYB:FORK, HYB:MERGE, HYB:COND_EXEC, HYB:REDUCE, HYB:ESTIMATE

Short demo time

<https://github.com/nunezco2/cqam>



**National Center for
Supercomputing Applications**

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN